

Correctness: **Author: Wenyu (Wendy) Fan**

- data structures → invariant (the invariant holds before an operation is executed, the operation has the desired result and the invariant still holds after the operation is executed)
- greedy → exchange argument (see 9-Greedy)
- divide and conquer → induction (recursion)

1 - Time

|              |                           |
|--------------|---------------------------|
| constant     | $T(n) = \Theta(1)$        |
| logarithmic  | $T(n) = \Theta(\log n)$   |
| linear       | $T(n) = \Theta(n)$        |
| quasi-linear | $T(n) = \Theta(n \log n)$ |
| quadratic    | $T(n) = \Theta(n^2)$      |
| cubic        | $T(n) = \Theta(n^3)$      |
| exponential  | $T(n) = \Theta(c^n)$      |

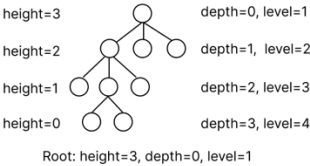
2 – Lists initialize -> O(1)

| Data Structure  | Operation                                                                 | Time        |
|-----------------|---------------------------------------------------------------------------|-------------|
| Array           | <code>get(i)</code> , <code>set(i, e)</code>                              | O(1)        |
|                 | <code>insert(0, e)</code> , <code>remove(0)</code> from the start         | O(n)        |
|                 | <code>insert(i, e)</code> , <code>remove(i)</code> in the middle          | O(n)        |
|                 | <code>append(e)</code> , <code>pop()</code> from the end                  | O(1)        |
|                 | <code>replace(i, e)</code> , <code>size()</code> , <code>isEmpty()</code> | O(1)        |
| Singly Linked   | <code>get(i)</code>                                                       | O(n)        |
| / Doubly Linked | <code>addFirst(e)</code> , <code>removeFirst()</code> from start          | O(1)        |
|                 | <code>add(i, e)</code> , <code>remove(i)</code> in middle                 | O(n)        |
|                 | <code>addLast(e)</code> , <code>removeLast()</code> from end              | O(n) / O(1) |
|                 | <code>replace(i, e)</code>                                                | O(n)        |
| Stack           | <code>push(e)</code> , <code>pop()</code>                                 | O(1)        |
|                 | <code>enqueue(e)</code> , <code>dequeue()</code>                          | O(1)        |

3 – Trees

Preorder, Inorder, Postorder → O(n)

| Method                                                                           | Time |
|----------------------------------------------------------------------------------|------|
| <code>size()</code> , <code>isEmpty()</code>                                     | O(1) |
| <code>root()</code> , <code>parent(p)</code>                                     | O(1) |
| <code>children(p)</code>                                                         | O(c) |
| <code>numChildren(p)</code>                                                      | O(1) |
| <code>isInternal(p)</code> , <code>isExternal(p)</code> , <code>isRoot(p)</code> | O(1) |



4 – Binary Search Tree should consider h's best & worst case

Binary Search Tree (BST)

- Properties:
  - For any node v, keys in the left subtree are less than v, and keys in the right subtree are greater than v.
  - In-order traversal of a BST results in keys in increasing order.
- Operations:
  - Search: `search(k, v)` O(h) (h = height)
  - Insertion: `put(k, o)` O(h)
    - If the key exists, replace the value.
    - Otherwise, insert at the reached external node.
  - Deletion: `remove(k)` O(h)
    - Case 1: Node has one external child — remove and promote the other child.
    - Case 2: Node has two internal children — find the in-order successor, copy its value, and delete it.
- Complexity:
  - Space: O(n)
  - Best Case Height: O(log n)
  - Worst Case Height: O(n)

Range Queries in BST

- Purpose: Find all keys k in range  $k_1 \leq k \leq k_2$ .
- Algorithm: in-order (ascending order)
  - Traverse left or right subtree based on key comparisons.
  - |inside nodes| ≤ |output|; |boundary node| ≤ 2 \* tree height
  - Time Complexity: O(|output| + log n)

AVL Tree (Self-Balancing BST)

nodes usually contains a key/a pointer

- Properties:
  - Balanced BST where the heights of left and right subtrees of every node differ by at most 1.
  - Height: O(log n)
- Operations:
  - Search:
    - Time Complexity: O(log n)
  - Insertion:
    - Standard BST insertion, followed by rebalancing.
    - If balance property is violated, perform rotations
    - Time Complexity: O(log n)
  - Deletion:
    - Standard BST deletion, followed by rebalancing using rotations if necessary.
    - Time Complexity: O(log n)

3. Trinode Restructuring:
- 
- Helps restore balance by making the middle node of three unbalanced nodes the root of that subtree.
  - Time Complexity: O(1) for rotation operations.

5 - Priority Queues

Priority queue implementations

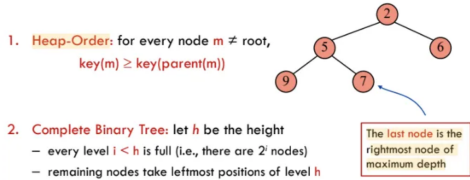
| Method        | Unsorted List | Sorted List | Heap     |
|---------------|---------------|-------------|----------|
| size, isEmpty | O(1)          | O(1)        | O(1)     |
| insert        | O(1)          | O(n)        | O(log n) |
| min           | O(n)          | O(1)        | O(1)     |
| removeMin     | O(n)          | O(1)        | O(log n) |
| remove        | O(1)          | O(1)        | O(log n) |
| replaceKey    | O(1)          | O(n)        | O(log n) |
| replaceValue  | O(1)          | O(1)        | O(1)     |

Priority Queue Sorting Algorithms

- Selection Sort (Unsorted Sequence):
  - Time Complexity: `insert` : O(n), `remove_min` : O(n^2)
- Insertion Sort (Sorted Sequence):
  - Time Complexity: `insert` : O(n^2), `remove_min` : O(n)
- Heap-Sort (Uses a min-heap for efficient sorting)
  - Time Complexity: `insert`, `remove_min` : O(log n)

Heap Data Structure (Min-Heap)

1. Properties: (remember to explain both nodes and the root of the heap)



2. Height: O(log n).
3. Min-Heap Operations:

| Operation                          | Description                                                                                         | Time     |
|------------------------------------|-----------------------------------------------------------------------------------------------------|----------|
| Build Heap                         | Builds a heap from an unsorted array                                                                | O(n)     |
| <code>insert</code>                | Insert at the last position and perform <b>upheap</b>                                               | O(log n) |
| <code>remove_min</code>            | Replace the root with the last node, remove the last node, and perform <b>downheap</b>              | O(log n) |
| Access root <code>min</code>       | Returns the min/max element without removal                                                         | O(1)     |
| Decrease/Increase Key              | Adjusts the value of a specific key                                                                 | O(log n) |
| Find Next Last Node After Deletion | Start from the old last node, go up to find a right child or root, go to sibling, then down to leaf | O(log n) |

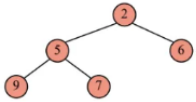
Heap Array Representation

Special nodes:

- root is at 0
- last node is at  $n-1$

For the node at index i:

- the left child is at index  $2i+1$
- the right child is at index  $2i+2$
- Parent is at index  $\lfloor (i-1)/2 \rfloor$



- Counting Inversions with Selection Sort: (of adjacent elements)
  - the number of swaps needed is equal to the number of inversions in the array.
- Finding the k-th Value in Sorted Order Using Heaps:
  - keep only the k largest elements in the min-heap by replacing the root if a new element is larger. After inserting k elements, the root will be the k-th largest element.
- Merging k Sorted Lists of Length m Using a Min-Heap:
  - For k lists, inserting and removing elements from the heap will take O(klogk) time per element added to the result list.
  - Overall Complexity: O(kmlogk)

6 – Hashing

Implementations of Map ADT

| Implementation        | Description                                              | Insert (put)   | Search (get)   | Delete (remove) | Space                    |
|-----------------------|----------------------------------------------------------|----------------|----------------|-----------------|--------------------------|
| Simple List-Based Map | Stores items in an unsorted list                         | O(1) (append)  | O(n)           | O(n)            | O(n)                     |
| Array-Based Map       | Uses an array for keys in a fixed range (e.g., [0, N-1]) | O(1)           | O(1)           | O(1)            | O(N) (for fixed range N) |
| Hash Table            | Maps keys to indices using a hash function               | O(1) (average) | O(1) (average) | O(1) (average)  | O(n)                     |

Collision Handling

| Method            | Description                                                                       | Average Time Complexity                                                                                  | Suitable Scenarios                                                    |
|-------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Separate Chaining | Each table cell points to a linked list that holds all items hashed to that index | O(1+α), worst-case O(n)                                                                                  | Allows higher load factors; requires extra memory for linked lists    |
| Linear Probing    | Places colliding items in the next available position (circularly)                | O(1) with low load factor, worst-case O(n) ( <code>get</code> , <code>put</code> , <code>remove</code> ) | Efficient with careful load factor control, but may cause clustering  |
| Cuckoo Hashing    | Uses two hash tables and two hash functions                                       | worse-case O(1) for <code>get</code> , <code>remove</code> , expected O(1) for <code>put</code>          | Offers worst-case guarantees for lookups; more complex implementation |

Eviction Cycle Detection:

- Use a Counter: to keep track of the number of evictions. If we evict enough times we are guaranteed to have a cycle.
- Flag Entries: Keep an additional flag for each entry. Every time we evict an entry, we flag it. After a successful put, we need to unflag the entries flagged.

|                                                                     |            |                             |                  |
|---------------------------------------------------------------------|------------|-----------------------------|------------------|
| • $n$ vertices, $m$ edges<br>• no parallel edges<br>• no self-loops | Edge List  | Adjacency List              | Adjacency Matrix |
| Space                                                               | $O(n + m)$ | $O(n + m)$                  | $O(n^2)$         |
| incidentEdges( $v$ )                                                | $O(m)$     | $O(\deg(v))$                | $O(n)$           |
| getEdge( $u, v$ )                                                   | $O(m)$     | $O(\min(\deg(u), \deg(v)))$ | $O(1)$           |
| insertVertex( $x$ )                                                 | $O(1)$     | $O(1)$                      | $O(n^2)$         |
| insertEdge( $u, v, x$ )                                             | $O(1)$     | $O(1)$                      | $O(1)$           |
| removeVertex( $v$ )                                                 | $O(m)$     | $O(\deg(v))$                | $O(n^2)$         |
| removeEdge( $e$ )                                                   | $O(1)$     | $O(1)$                      | $O(1)$           |

if Adjacency List need traverse, `removeVertex`:  $O(n+m)$  `removeEdge`:  $O(m)$

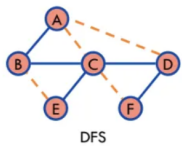
Graph Traversal

| Algorithm | Complexity (Adjacency List) | Edges       |
|-----------|-----------------------------|-------------|
| DFS       | $O(n+m)$                    | back edges  |
| BFS       | $O(n+m)$                    | cross edges |

| Applications                                         | DFS | BFS |
|------------------------------------------------------|-----|-----|
| Spanning forest, connected components, paths, cycles | ✓   | ✓   |
| Shortest paths                                       |     | ✓   |
| Cut edges                                            | ✓   |     |

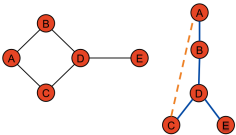
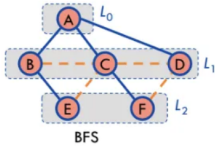
Non-tree DFS edge ( $v, w$ )  
 $w$  is an ancestor of  $v$  in the DFS tree

Called **back edges**



Non-tree BFS edge ( $v, w$ )  
 $w$  is in the same level as  $v$  or in the next level

Called **cross edges**



2. Bipartite Check: Using BFS to color each node per layer.

- **Bipartite:** Adjacent nodes in different layers.

3. Cycle Detection

- Using Adjacency List:  $O(n)$ 
  - Perform DFS to mark each node.
  - If you revisit a marked node, a cycle exists.
- Using Adjacency Matrix:  $O(n)$ 
  - Perform DFS to mark nodes by rows and columns.
  - Check in-degrees and out-degrees to avoid getting stuck.

4. Identifying Cut Vertices and Cut Edges:  $O(n+m)$

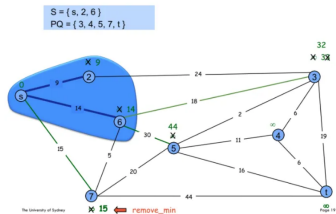
- **Cut Edges (Bridges):**
  - In a DFS tree, for an edge  $(u,v)$  where  $u$  is the parent and  $v$  is the child:
    - If `down-and-up[v] > level(u)`,  $(u,v)$  is a cut edge.
    - If `down-and-up[v] ≤ level(u)`,  $(u,v)$  is not a cut edge.
- **Cut Vertices (Articulation Points):**
  - The root of the DFS tree is a cut vertex if it has two or more children.
  - For any internal node  $u$  with two or more children,  $u$  is a cut vertex if and only if `down-and-up[v] ≥ level(u)` for any child  $v$ .

8 - Graph Algorithms sometimes modify adjacency list

Dijkstra's Algorithm

Standard Heap:  $O(m \log n)$ ; Fibonacci Heap:  $O(m + n \log n)$ ; Binary Heap:  $O((m+n) \log n)$

- **Algorithm Overview:**
  - Initialize the starting node distance as 0, and all other nodes as infinity.
  - Update distances of adjacent nodes accordingly while expanding the shortest path.

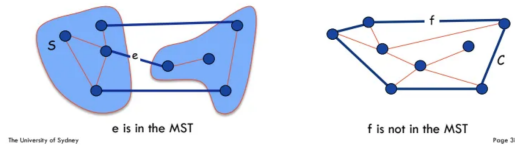


Minimum Spanning Tree

**Simplifying assumption.** All edge costs  $c_e$  are distinct.

**Cut property.** Let  $S$  be any subset of nodes, and let  $e$  be the **min cost edge with exactly one endpoint in  $S$** . Then the **MST contains  $e$** .

**Cycle property.** Let  $C$  be any cycle, and let  $f$  be the **max cost edge belonging to  $C$** . Then the **MST does not contain  $f$** .



Cycle-Cut Intersection

Claim: A cycle and a cutset intersect in an even number of edges.

Prim's Algorithm

- **Steps:**
  1. Initialize the MST with an arbitrary vertex.
  2. Add the minimum-weight edge connecting the current tree to any vertex not in the tree.
  3. Repeat until all vertices are included in the MST.
- **Time Complexity:**
  - Binary heap:  $O(m \log n)$ ; Fibonacci heap:  $O(m + n \log n)$ .

Kruskal's Algorithm

- **Steps:**
  1. Sort all edges by weight.
  2. For each edge, check if it forms a cycle with the current MST using Union-Find.
  3. If no cycle is formed, add the edge to the MST.
  4. Stop once  $n-1$  edges are added.
- **Time Complexity:**
  - Sorting edges:  $O(m \log m)$ .
  - Using Union-Find to check cycles:  $O(n^2)$
  - `make_sets(A)`:  $O(n)$  `find(u)`:  $O(1)$  `union(u, v)`:  $O(n)$

Shortest Path Problem with Edge Weights and Vertex Costs

- Create a new graph  $G'$  by modifying each edge  $(u,v)$  in  $G$ .
- In  $G'$ , set the weight of  $(u,v)$  to **edge weight + cost of the destination vertex**.
- Run Dijkstra's Algorithm on  $G'$

9 -Greedy Algorithms

Key Problems and Algorithms

| Problem               | Greedy Choice                                                                                | Complexity                                                             | Correctness                            |
|-----------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------|
| Fractional Knapsack   | Maximize benefit within a weight constraint by taking fractions of items                     | $O(n \log n)$ for sorting, $O(n)$ for processing                       | Optimal (exchange argument)            |
| Interval Partitioning | Minimize the number of classrooms needed for lectures, ensuring no overlap in the same room. | $O(n \log n)$ using a priority queue                                   | Optimal (uses priority queue)          |
| Huffman Encoding      | Combine lowest frequency characters                                                          | $O(n \cdot d \log d)$ , where $d$ is the number of distinct characters | Optimal (builds minimum encoding tree) |

**Theorem:** The greedy strategy of picking item with highest benefit to weight ratio computes an optimal solution.

**Proof (sketch):**

- Use an exchange argument
- Assume for simplicity that all ratios are different  $b_i/w_i \neq b_k/w_k$
- Consider some feasible solution  $x$  different than the greedy one
- There must be items  $i$  and  $k$  s.t.  $x_i < w_i$ ,  $x_k > 0$  and  $b_i/w_i > b_k/w_k$
- If we replace some  $k$  with some of  $i$ , we get a better solution
- How much?  $\min\{w_i - x_i, x_k\}$
- Thus, there is no better solution than the greedy one

**Thm:** Huffman's algorithm computes a minimum length encoding tree of  $(C, f)$

10 - Divide and Conquer

| Example Algorithm   | Recurrence                    | Solution                 |
|---------------------|-------------------------------|--------------------------|
| Array Binary Search | $T(n) = T(n/2) + O(1)$        | $O(\log n)$              |
| Merge Sort          | $T(n) = 2T(n/2) + O(n)$       | $O(n \log n)$            |
| Quick Sort          | $T(n) = T(nL) + T(nR) + O(n)$ | $O(n \log n)$ on average |

Proof by Unrolling -  $T(n) = 2T(n/2) + O(n)$

Hence, the recursion is:

$$T(n) = \begin{cases} T(n) = 2T(n/2) + O(n) & \text{for } n > 1 \\ O(1) & \text{for } n = 1 \end{cases}$$

which solves to

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n) = c \cdot n + 2 \cdot c \cdot \frac{n}{2} + 4 \cdot c \cdot \frac{n}{4} + \dots = O(n \log n)$$

**Simplifying the Series:** The recursive term  $T\left(\frac{n}{2^k}\right)$  reaches the base case when  $\frac{n}{2^k} = 1$ , which implies  $k = \log_2(n)$ . Substituting  $k = \log_2(n)$  gives:

$$T(n) = n \cdot T(1) + c \cdot n \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n}\right)$$

Some recurrence formulas with solutions

| Recurrence                    | Solution             |
|-------------------------------|----------------------|
| $T(n) = 2 T(n/2) + O(n)$      | $T(n) = O(n \log n)$ |
| $T(n) = 2 T(n/2) + O(\log n)$ | $T(n) = O(n)$        |
| $T(n) = 2 T(n/2) + O(1)$      | $T(n) = O(n)$        |
| $T(n) = T(n/2) + O(n)$        | $T(n) = O(n)$        |
| $T(n) = T(n/2) + O(1)$        | $T(n) = O(\log n)$   |
| $T(n) = T(n-1) + O(n)$        | $T(n) = O(n^2)$      |
| $T(n) = T(n-1) + O(1)$        | $T(n) = O(n)$        |

11 - Divide and Conquer II

Master Theorem

Let  $f(n)$  and  $T(n)$  be defined as follows:

$$T(n) = \begin{cases} a T(n/b) + f(n) & \text{for } n \geq d \\ c & \text{for } n < d \end{cases}$$

Depending on  $a$ ,  $b$  and  $f(n)$  the recurrence solves to:

1. if  $f(n) = O(n^{\log_b a - \epsilon})$  for  $\epsilon > 0$  then  $T(n) = O(n^{\log_b a})$ ,
2. if  $f(n) = O(n^{\log_b a} \log^k n)$  for  $k \geq 0$  then  $T(n) = O(n^{\log_b a} \log^{k+1} n)$ ,
3. if  $f(n) = O(n^{\log_b a + \epsilon})$  and  $a f(n/b) \leq \delta f(n)$  for  $\epsilon > 0$  and  $\delta < 1$  then  $T(n) = O(f(n))$ ,

Note: If  $f(n)$  is given as big- $O$ , you can only conclude  $T(n)$  as big- $O$  (not  $\Theta$ ).

Given a recursive algorithm with time complexity recurrence:  $T(n) = a \cdot T\left(\frac{n}{b}\right) + n^d$ ,

- When  $a < b^d$  (i.e.,  $\log_b a < d$ ),  $T(n) = O(n^d)$ .
- When  $a = b^d$  (i.e.,  $\log_b a = d$ ),  $T(n) = O(n^{\log_b a} \cdot \log n)$ .
- When  $a > b^d$  (i.e.,  $\log_b a > d$ ),  $T(n) = O(n^{\log_b a})$ .